

EvoDRC: A Self-Evolving Agentic Framework for Automated DRC Violation Repair

Bing-Yue Wu¹, Chia-Tung Ho², Haoyu Yang², Brucek Khailany², and Vidya A. Chhabria¹

¹Arizona State University; ²NVIDIA Corporation

Abstract

Design rule check (DRC) closure remains a major bottleneck in advanced-node physical design. Although detailed routers are rule-aware, residual design rule violations (DRVs) often require manual engineering change order iterations. Automating this process is challenging because repairs must account for complex geometric interactions, preserve circuit connectivity, and avoid introducing new violations. We present EvoDRC, a skill-evolution framework for agentic block-level DRC repair. EvoDRC initializes layer-specific repair skills using knowledge distilled from an unrelated reference design and continuously evolves these skills using traceable repair experience collected from the target design. EvoDRC decomposes the layout into bounded repair regions and assigns an LLM repair agent to each region. Local DRC analysis, connectivity-checking, and impact-preview tools provide feedback on proposed modifications. Repair operations and their resulting DRV changes are stored in a knowledge database and used to evolve the repair skills. Experiments on seven block-level designs from the DAC26 DRC Benchmark show that EvoDRC achieves a 73.5% overall reduction compared to the reported baseline.

1 Introduction

Design rule check (DRC) closure remains a critical bottleneck in advanced-node physical design. Foundry design rules impose increasingly complex geometric constraints involving metal width, spacing, enclosure, alignment, and interactions across multiple routing layers. Although modern detailed routers are rule-aware, they do not always produce DRC-clean layouts. Residual design rule violations (DRVs) must therefore be resolved through engineering change order (ECO) iterations, which often require substantial manual effort from layout engineers. Engineers must inspect each violation in its surrounding layout context, identify the relevant design rule, determine its root cause, and apply a repair without disrupting circuit connectivity or introducing additional violations. This process is inherently iterative because a local geometric edit can produce ripple effects elsewhere in the layout. Consequently, post-route DRC repair can consume significant engineering effort. Industry reports indicate that iterative DRC closure can require multiple ECO and verification cycles over several days or weeks [1].

Large language models (LLMs) have recently demonstrated strong capabilities as agents that interact with external tools, reason over intermediate results, and refine their actions in a closed loop [2]. Recent studies in physical design use agentic systems to generate scripts and interact with EDA tools for automating placement, routing, and timing optimization [3–8]. These results suggest that LLM agents can automate key physical design stages. However, agentic DRC repair remains largely unexplored. Existing studies apply LLMs to design-rule interpretation and DRC checker code generation [9], while recent efforts demonstrate agentic DRC repair on

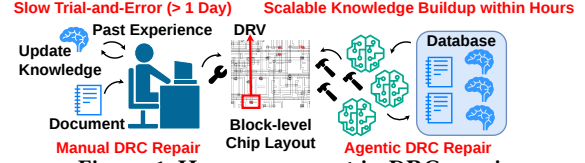


Figure 1: Human vs agent in DRC repair.

small-scale tasks, such as standard-cell-level layouts [10, 11]. These methods are difficult to scale to block-level complex designs with diverse design rules spanning multiple routing layers. Moreover, an LLM agent’s repair capability strongly depends on a well-developed, task-specific skill file that encodes factual repair knowledge, but such a skill file is rarely available for a new target design.

In practice, a DRC engineer often performs similar repair tasks across multiple designs, as shown in Fig. 1. When a familiar violation pattern appears, the engineer can reuse repair strategies learned while transforming previous DRC-violating designs into DRC-clean designs. However, a new layout may expose previously unseen rule interactions or geometric contexts, requiring the engineer to refine existing strategies or learn new ones. We formulate this process as a *skill-evolution* task: repair experience from a previous design provides an initial set of transferable skills, while verified repair outcomes from the target design are used to continuously evolve those skills. In this work, we present **EvoDRC**, a skill-evolution framework targeting block-level DRC repair. EvoDRC initializes its repair skills by distilling knowledge from transforming an unrelated DRC-violating reference design into a DRC-clean design. It subsequently refines these skills using traceable repair experience collected during repairs to the target design. To improve scalability, EvoDRC decomposes large layouts into bounded *repair crops* and dispatches one repair agent per crop. Each agent explores repair strategies using preview tools that provide feedback on local DRC changes, circuit connectivity, and the impact of a proposed edit. EvoDRC stores approved repair operations together with their resulting changes in a knowledge database (Knowledge DB). It then evolves layer-specific skill files over successive repair iterations by distilling both successful and unsuccessful repair records. This process combines cross-design skill transfer with test-time skill evolution while preserving a traceable connection between each learned skill and its repair outcomes. The contributions are:

- To the best of our knowledge, EvoDRC is the first agentic DRV repair framework that automates traditionally manual post-route DRC ECOs, allowing layout engineers to focus on other tasks.
- EvoDRC formulates DRC repair as a skill-evolution task. By combining skill transfer from a reference design with skill evolution on the target design, EvoDRC continuously improves the repair knowledge provided to its agents.
- EvoDRC introduces a layout-decomposition mechanism that improves the scalability of agentic DRC repair by dividing block-level layouts into bounded repair crops.

- Experimental results on block-level test cases from the DAC26-DRC-Benchmark [12] show that EvoDRC reduces DRV by 73.5%. EvoDRC is available at [13].

2 Related work

2.1 Placer- and Router-Driven DRC Reduction

Most existing efforts focus on reducing DRVs during the placement and routing stages of the physical design flow. At the placement stage, predictive methods estimate DRV hotspots before routing and reshape the layout through methods such as synthesized routing blockages, whitespace redistribution, or routing resource adjustment [14–16]. However, these techniques still leave residual DRVs that must be repaired manually. At the routing stage, rule-aware detailed routers integrate DRC into itself through pin-access analysis, track assignment, and adaptive rip-up [17, 18]. In practice, the router checks a simplified rule model instead of the sign-off deck for scalability, so layouts still have residual DRVs. After routing, ECO-stage optimizers repair residual DRVs by jointly refining cell placement, routing wires, and cell selection within a local region under satisfiability modulo theories (SMT). [19, 20]. However, each rule must be hand-translated into constraints, so these optimizers do not extend to a full sign-off deck.

2.2 LLMs for DRC-related tasks

Recent works have explored the application of LLMs to DRC-related tasks. Existing studies use LLM agents with multimodal design-rule interpretation, code execution, and debugging feedback to translate design-rule descriptions into executable DRC checker code [9]. The results demonstrate that LLMs can read design-rule descriptions, relate them to layout geometry, and produce correct checking logic. However, generating a checker addresses violation detection rather than violation repair. In DRC checker code generation, the agent neither modifies the layout nor encounters the ripple effects of geometric edits. Agentic DRC repair, therefore, requires a different interface, in which the agent proposes layout operations and an external framework verifies their consequences.

2.3 DRC Repair Benchmarks

Open-source benchmarks for DRC repair have only recently appeared [10–12, 21]. One representative benchmark, the DAC26-DRC-Benchmark [10, 12], releases test cases at multiple scales, including polygon-level, standard-cell-level, and block-level cases, to evaluate both the capability and scalability of LLM workflows on DRC repair problems of varying scale of problem. Implemented with the ASAP7 PDK [22], this benchmark provides each test case with the layout GDSII, a layout script, a DRC report from KLayout, the open-source DRC tool [23], and a net connectivity checker. Its experiments provide an agentic DRC repair workflow with the layout GDSII, layout script, design rule deck, design rule manual (DRM) information, and KLayout tool. The results show that many available LLMs exhibit strong reasoning capability on standard-cell-level DRC repair tasks. However, repair quality degrades drastically when the same workflow is applied to block-level layouts. This indicates the need for a scalable approach to deploying agentic DRC repair on block-level cases. Another benchmark, PostEDA-Bench [11, 21], provides checkable tasks for residual sign-off DRC repair and reveals a substantial capability gap between synthetic rule-level exercises and practical geometric, multi-step DRC reasoning. Both benchmarks

provide evaluation harnesses, making agentic DRC repair directly measurable and reproducible. In this work, we evaluate EvoDRC on the block-level test cases from the DAC26-DRC-Benchmark rather than PostEDA-Bench, since PostEDA-Bench is limited to a single violation and its DRC-fixing tasks are restricted to small designs with at most 14 DRVs, whereas the DAC26-DRC-Benchmark blocks contain up to 765 DRVs.

3 The EvoDRC Framework

Fig. 2 illustrates the overview of the EvoDRC framework. First, in the layout decomposition stage (left top corner of Fig. 2), given a layout and its DRC report, EvoDRC classifies different routing types and then crops the layout based on the DRV distribution for each routing type to form *repair crops*. Second, in the agentic repair stage, DRC repair agents are dispatched to work on each repair crop. Additional tools are provided to assist the repair process by informing the agents of the consequences of intermediate repair trials, including intermediate DRV distribution changes, net connectivity preservation, and the impact range of each action. The DRC repair agents are provided with the assigned repair crop and the DRV distribution of that crop, DRM images as visual cues for the design rules, and layer-based skill files that document the design rule deck used to run DRC analysis and the DRC repair knowledge used to guide layer-wise DRC repair. The agents adopt the ReAct paradigm [2] to reason and act on DRC repair.

In the third stage, after the agents provide the suggested repair operations, EvoDRC examines whether the operations break circuit logic and decides which operation to take when conflicting operations from different agents exist. After examination, the accepted repair operations are applied to each corresponding repair crop. EvoDRC then runs DRC on each crop and records the operation-result pairs into the knowledge database (Knowledge DB). In the Knowledge DB, two agents, i.e., the Skill Refiner and the DB Summarizer, are dispatched to generate candidate skill files for each layer that may be used in the next iteration of DRC repair through two different approaches. The generated skill files are then passed through a script-based checker to ensure quality. The Skill Judge agent then takes the candidate skill files and decides whether the current skill file in the Knowledge DB should be replaced. EvoDRC-driven DRC repair then proceeds to the next iteration with the updated skill file. EvoDRC is designed to refine and evolve the skill file that guides DRC repair over iterations, helping DRC repair agents achieve higher repair quality.

3.1 Layout Decomposition

As shown in Stage 1 in Fig. 2, EvoDRC first parses the design and classifies different routing types. For each routing type, EvoDRC builds layout crops based on the DRV distribution from the DRC report. Regular metal-wire polygons and vias that touch or overlap the DRV region are used to enlarge the cropped layout by computing the minimum and maximum vertex coordinates in the x and y directions to form a first-order crop. To prevent a long stripe from expanding a crop to an unnecessarily large crop, EvoDRC bounds the first-order crop to a size of $k \mu\text{m} \times k \mu\text{m}$. First-order crops that overlap in x and y coordinates and metal layers are merged into second-order crops. EvoDRC then unions second-order crops that fit within the same standard-cell-height row to form final *repair crops*, while leaving each multi-row crop as an independent repair

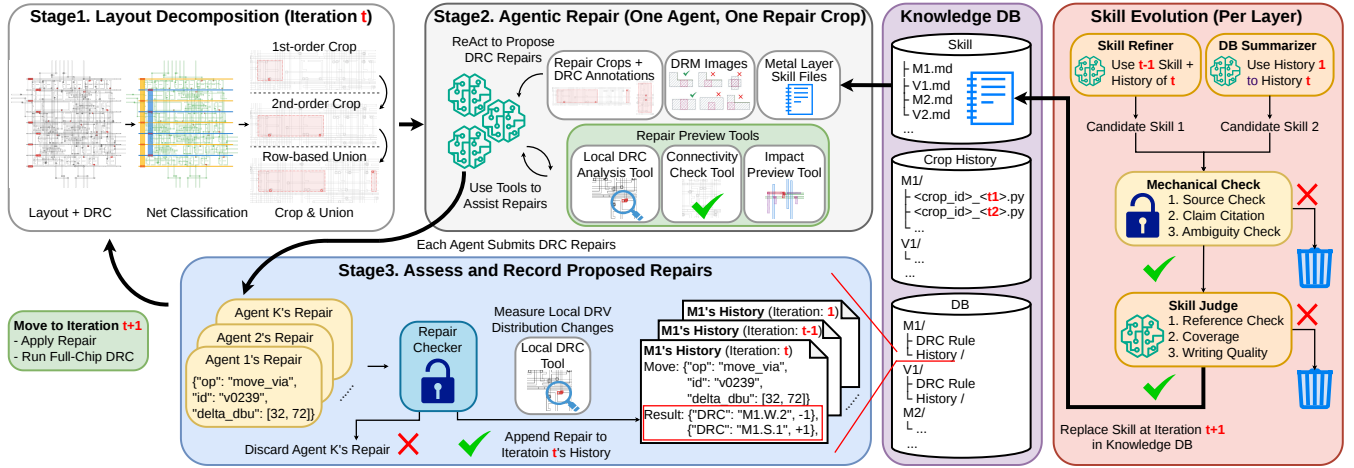


Figure 2: Overview of EvoDRC. The framework consists of layout decomposition, agentic DRC repair, and skill evolution. Stage 1 decomposes the layout into repair crops based on the DRV distribution (marked in red). Stage 2 dispatches agents to repair each crop using preview tools, DRM images, and layer-based skill files. Stage 3 records validated operation-result pairs in the Knowledge DB to evolve skills for the next iteration.

Table 1: Object editability classes and available operations.

Class	Object type	Operation	Applicable scope
A	Via structure	move_via	Move a via
		delete_via	Delete a via
		add_via	Add a via
		resize_via_shape	Resize a metal polygon within a via structure
		move_via_shape	Move a metal polygon within a via structure
B	Polygon fully within the region	resize	Resize a polygon
		move	Move a polygon
		delete	Delete a polygon
		add_vertex	Modify the shape of a polygon
		add_polygon	Add a polygon
C	Polygon cropped by the k μm window	resize_end	Shorten or elongate the open end
D	Standard cell and polygons of other routing types		No repair operation allowed

Table 2: Preview tools available during repair.

Tool	Returned information
Local DRC analysis	Run DRC on the selected repair region in a sandbox for fast region-local feedback. Reports whether the target DRV is cleared, remaining targets, and new DRVs with bounding boxes
Connectivity check	Result of the full-design connectivity check and the seeds of broken nets
Impact preview	Metal/via objects touching or overlapping an edited object, including each neighbor’s layer, kind, and editability

crop. The resulting repair crops are then passed individually to the repair agents in parallel.

3.2 Agentic Repair

As Stage 2 in Fig. 2 illustrates, for each repair crop, the repair agent receives (i) design rule manual (DRM) images, (ii) the repair crop represented by a KLayout Python script, (iii) its DRV distribution, i.e., DRV type and location, and (iv) layer-wise skill files for the layers spanned by the repair crop. Each layer-wise skill file combines the corresponding KLayout DRC deck describing the design rules with repair knowledge distilled from previous repair iterations.

EvoDRC assigns each metal polygon or via structure in the repair crop to one of the four classes in Table 1: (i) Vias in the repair crop can be moved, deleted, or added only as complete objects. No partial edits may be applied to a single via. Any modification to a via structure must be applied to all vias in the whole layout. (ii) Metal stripes fully within the repair crop are allowed to have any operation applied to them, such as shape modification, moving,

deleting, or creating new metal stripes. (iii) Cropped metal stripes in the repair crop are restricted to elongating or shortening their open ends within the repair crop to prevent unexpected consequences in unseen regions of the layout. (iv) Standard cells internal polygons are not allowed to be modified to prevent degradation of the circuit logic, as are metal stripes and vias that belong to other routing types. These classes distinguish editable objects from frozen context and prevent a crop boundary from implicitly granting control over an unseen portion of the design. Table 1 defines the available operations for each class of object.

To assist the repair agent in its ReAct-based agentic repair, three preview tools in Table 2 are provided to each repair agent: (i) The local DRC analysis tool can be used to run DRC analysis on the repair crop and show the DRV distribution difference during repair operation exploration, which helps the repair agent reason about different repair strategies. (ii) The connectivity check tool can verify whether the proposed repair operation breaks net connectivity, which affects circuit logic, so such operations are forbidden¹. (iii) The impact preview tool is used to provide the possibly affected objects when the agent tries to apply operations to objects in the repair crop. Each repair agent repairs the assigned repair crop using the provided input and tools, and returns a list of repair operations.

3.3 Assess and Record Proposed Repairs

EvoDRC accepts a repair agent’s final repair operation list only when it passes the full-design connectivity check. The framework applies this list of repair operations to the repair crop and runs local DRC on the repair crop to record the local DRV distribution changes. When conflicting operations are proposed by different repair agents, e.g., when two repair agents propose different approaches to modifying the structure of the same generated via, the same edit on the same object may have different impacts in different contexts because each repair agent only reasons about and proposes repair operations to resolve local DRVs. Therefore, for repair crops whose agents propose structure edits on the same

¹This check is not an LVS run. It applies the candidate operations to a copy of the layout and reruns the DAC26-DRC-Benchmark connectivity checker, a simple depth-first search over the merged metal-via stack that verifies each standard-cell pin still reaches its corresponding standard-cell or IO pins.

generated via, EvoDRC applies the via edits individually and tests which via edit performs best, i.e., yields the most net DRV count reduction or the smallest DRC increase.

Finally, EvoDRC groups the repair operation–DRV distribution change records by metal layer and appends them to the Knowledge DB, as shown in Stage 3 in Fig. 2. Each record includes the repair operations on each layer and their corresponding DRV distribution changes on the same metal layer, preserving trackable results rather than blindly trusting an agent’s output. The geometry of repair crops is also stored in the Knowledge DB for future replay purposes.

3.4 Knowledge DB

The key component of EvoDRC is the Knowledge DB, which not only stores the skill files for all layers used in the agentic DRC repair process but also keeps the proposed repair operations and DRV distribution changes within the repair crop, as well as the geometry of the repair crops. The Knowledge DB stores these records by metal layer and iteration timestamp.

3.4.1 Knowledge DB Organization The purple box in Fig. 2 summarizes the database organization. The Knowledge DB stores three types of data: (i) the DB directory contains storage directories for each metal layer, such as M1, M2, and V1. Each directory stores the Design Rule, which is the corresponding KLayout DRC deck for that layer describing the design rule, and the History directory, which stores all repair operation–local DRV distribution change records. (ii) the Skill directory stores the skill file for each metal layer, while the Crop History directory preserves repair crop geometry by layer, crop, and iteration annotation, serving as a backup to replay the geometry of repair crops from previous iterations.

3.4.2 Trackable Records EvoDRC is intentionally designed to have each repair agent return only its final proposed repair operations in order to keep all records trackable. Intermediate trials are often untraceable because they may start from temporary geometries produced by earlier speculative edits during the agents’ DRC repair exploration. Simply applying such operations to the original geometry can cause them to be accumulated as noisy data and progressively bias the Knowledge DB. As a result, EvoDRC discards all intermediate trial records and only allows each repair agent to return its final verdict, which is then stored in the Knowledge DB.

3.5 Skill Evolution

After the operation–DRV distribution change records of the current DRC repair iteration are stored, EvoDRC launches two skill-evolution agents independently for each metal layer with new records, as shown on the right of Fig. 2. (i) The Skill Refiner reads the Design Rule and the skill file for the corresponding layer from the Knowledge DB and combines them with the records collected in the current iteration. (ii) The DB Summarizer reads the complete history of that layer across all iterations, along with the Design Rule from the Knowledge DB, to construct a skill file for that layer. The Skill Refiner incrementally refines the existing skill files using the repair operation and DRV distribution change records collected in the current iteration, whereas the DB Summarizer reconstructs a new skill file from that layer’s complete repair history. Both knowledge skill candidates are evaluated against the same metrics described in Sections 3.5.1 and 3.5.2. Example skill files are at [13].

3.5.1 Mechanical Check After the Skill Refiner and the DB Summarizer each generate a skill file, EvoDRC applies three script-based checks to each skill file candidate:

- (1) Source check: Every reference must exist in the layer’s History directory in the Knowledge DB.
- (2) Claim citation: Every paragraph in the skill candidate having assertive words such as “do not,” “never,” or “always,” must at least cite one existing reference in the layer’s History directory in the Knowledge DB.
- (3) Ambiguity check: Must not use uncertain words such as “likely,” “maybe,” “might,” or “untested.”

Candidates that fail any check are discarded. These checks enforce minimum traceability rather than factual correctness.

3.5.2 Skill Judge When both skill candidates pass the mechanical checks, the Skill Judge agent compares them using three metrics:

- (1) Reference check: Each statement must be backed by at least one repair record.
- (2) Coverage: Prefers the candidate that accounts for more of the layer’s repair records, weighting the newest most.
- (3) Writing quality: Favors the candidate with less redundancy, clearer wording, and fewer contradictions.

The candidate receiving the majority of the three votes is selected. If only one candidate passes the mechanical checks, the Skill Judge decides whether it should replace the current skill file. Otherwise, the existing skill is retained. The selected skill is stored in the Skill directory and used in iteration $t + 1$. EvoDRC then performs full-layer DRC analysis and advances to iteration $t + 1$, where the updated skill is provided to the repair agents.

4 Experimental Setup

4.1 Benchmark and Test Cases

To evaluate the scalability and effectiveness of the EvoDRC framework, we conduct experiments on the seven block-level test cases from the DAC26-DRC-Benchmark [12]. All cases are implemented with the ASAP7 PDK [22]. Each test case provides the layout in the form of a GDSII file and a KLayout Python script, a KLayout DRC report, and net connectivity checker file of the design. We follow the DAC26-DRC-Benchmark and use KLayout 0.30.1 [23] with the provided ASAP7 KLayout rule deck. The seven blocks span 68 to 765 initial DRVs, covering metal width, spacing, area, enclosure, and grid-alignment constraints. In addition to using these seven blocks as test cases, we implement an external design, “carry_lookahead_adder” (CLA), from OpenCores [24] using OpenROAD-flow-scripts [25] and run DRC. Table 3 shows the dimension and scale differences among all eight cases, and Table 4 shows the DRC comparison across the cases. To the best of our knowledge, no prior work has attempted automatic, ECO-style DRC repair at this scale, and the DAC26-DRC-Benchmark, whose block-level design contains upto 765 DRVs, is the largest DRC repair benchmark available.

As shown in Table 3 and Table 4, CLA is significantly smaller than the seven blocks in DAC26-DRC-Benchmark, and its initial DRV distribution differs substantially from theirs. We manually remove the DRVs in the CLA block and use the DB Summarizer agent described in Sec. 3.5 to generate the initial skill file, which serves as the input in the first iteration of DRC repair for all the seven test cases in the following experiments.

Table 3: Benchmark and CLA design characteristics.

Design	#Insts.	Nets	Die area (μm^2)	Core area (μm^2)	Die $W \times H$ (μm)	Core $W \times H$ (μm)
Block1	143	120	16.06	9.30	4.008×4.008	3.132×2.970
Block2	62	43	8.99	4.08	2.998×2.998	2.160×1.890
Block3	76	62	10.07	5.02	3.174×3.174	2.322×2.160
Block4	107	81	13.45	7.58	3.668×3.668	2.808×2.700
Block5	47	28	7.27	2.97	2.696×2.696	1.836×1.620
Block6	155	101	17.94	11.02	4.236×4.236	3.402×3.240
Block7	574	410	57.91	43.74	7.610×7.610	6.750×6.480
CLA	52	229	4.818	2.712	2.195×2.195	1.674×1.620

Table 4: Initial DRV distribution by rule type.

Rule	Block1	Block2	Block3	Block4	Block5	Block6	Block7	CLA
M4.AUX.X	20	7	7	13	6	22	63	7
V1.M1.EN.1	11	2	6	4	5	10	26	7
V3.M4.AUX.2	–	–	–	–	–	–	–	7
V0.M1.AUX.3	37	12	21	20	9	21	97	6
M1.S.X	14	2	6	8	8	17	94	5
M5.AUX.X	8	4	4	6	4	8	12	4
M4.S.X	4	1	–	3	–	–	–	1
V2.M3.AUX.2	72	24	27	51	21	78	225	–
V4.M5.AUX.2	48	16	18	34	14	52	150	–
V5.M6.AUX.2	24	–	–	6	–	32	72	–
M3.S.X	2	–	–	1	1	3	15	–
M6.AUX.X	3	–	–	1	–	4	6	–
M2.S.X	1	–	–	–	–	–	4	–
Rule types	12	8	7	11	8	10	12	7
Total DRVs	244	68	89	147	68	247	765	37

We group DRCs of the same type under a shared label, e.g., M2.S.X is spacing rules on M2, and M5.AUX.X is auxiliary rules on M5. Hyphen implies no violation of that type

4.2 Experiment Configuration

We use Claude Sonnet 4.6 from Anthropic as the agent [26, 27] for all agents in the EvoDRC framework and set the reasoning effort to medium in the following DRC repair experiments. For the repair agent, it follows the ReAct paradigm and internally executes the four-role workflow: a planner, an adversarial plan reviewer, a DRC engineer, and a patch reviewer. As described in Sec. 3.2, each repair agent reads the assigned KLayout Python script-described repair crop, its DRV distribution, DRM images, and layer-wise skill files, and may use the three preview tools to assist DRC repair. There’s no runtime limitation for agents to repair DRC, a repair task completes when the agent submits its final repair operation list. Example prompts for all agents can be found at [13].

We allow EvoDRC to run for at most five iterations and stop early when the block reaches zero violations. For the crop-size restriction described in Sec. 3.1, we set it to $2\mu\text{m} \times 2\mu\text{m}$. We quantify DRC repair quality using % improvement in the net number of DRVs:

$$\text{Improvement} = \frac{\text{DRV}_{\text{initial}} - \text{DRV}_{\text{final}}}{\text{DRV}_{\text{initial}}} \times 100 \quad (1)$$

Here, $\text{DRV}_{\text{initial}}$ is the initial number of DRVs for each block, as shown in Table 4, and $\text{DRV}_{\text{final}}$ is the final number of DRVs after the experiment. As ablation, we compare three configurations each of which isolate one mechanism of EvoDRC:

- **Ablation 1:** Remove the CLA-derived skill files in the first iteration, leaving only the KLayout DRC deck. This evaluates whether the initial skill files contribute to repair quality.
- **Ablation 2:** EvoDRC using CLA-derived skill files for all iterations, with skill evolution disabled. This setup evaluates whether skill evolution contributes to repair quality.
- **Ablation 3:** EvoDRC using CLA-derived skill files for the first iteration, with the Layout Decomposition stage disabled. This setup evaluates whether decomposing a large design into small repair crops contributes to repair quality.

5 Results

5.1 Overall Repair Quality and Cost

Table 5 shows the DRV count changes over iterations for EvoDRC and the three ablation studies. Δ indicates the change in DRV count relative to the initial count, % shows this change as a % of the initial count. EvoDRC achieves an improvement of at least 55.3% on all blocks, fully cleans Block3 and Block5, and reaches an 83.6% improvement on average across all seven blocks, compared with 51.0%, 77.9%, and 73.0% for Ablations 1–3. Table 6 breaks down the EvoDRC dollar cost by agent role, using the Claude API pricing in effect at the time of publication [28]. Compared with manually fixing hundreds of residual DRVs which can take a DRC engineer days, this is a small repair cost.

Ablation 1 confirms the value of skill transfer. Without the CLA-derived initial skill files, EvoDRC’s improvement advantage reaches 77.9% on Block2 and exceeds 20% on every block except the DRC-clean Block3. On Block2, the ablation ends with more DRVs than it reaches at iteration 1. Because CLA is an external OpenCores design that is much smaller than the benchmark blocks and has a substantially different initial DRV distribution, the gap suggests that CLA-derived skills still help the first repair iterations on these targets. Ablation 2 confirms the value of skill evolution. With frozen skill files, its improvement is lower than EvoDRC on all six blocks that are not fully cleared, and its repair effectiveness drops in the late iterations. For example, Ablation 2 leaves two DRVs on Block5 while EvoDRC reaches a DRC-clean state, and it still leaves 358 DRVs on Block7 while EvoDRC further reduces the count to 275. Ablation 3 confirms that layout decomposition is essential for robust and scalable block-level repair. On Block7, EvoDRC removes 490 of 765 DRVs, whereas Ablation 3 removes only 81. This gap occurs because a single agent cannot cover enough violations within each iteration and leaves most violations unattempted. Repairing the entire design is competitive only on the smaller blocks. Therefore, decomposition is the mechanism that preserves EvoDRC’s repair coverage across design scales, while the transferred initial skills and skill-evolution mechanisms determine the quality of repair.

5.2 Skill Evolution Dynamics

In practice, a DRC engineer often reuses repair strategies learned from earlier designs when a familiar violation pattern reappears, but must refine those strategies, or learn new ones, when a new layout exposes unseen rule interactions or geometric contexts. EvoDRC formulates this process as skill evolution. Experience from a previous design provides transferable initial skills, while verified repair outcomes on the target design continuously evolve them.

Each entry in Table 7 reports the majority winner across all metal-layer skill updates for a specific experimental setting, block, and iteration. The winner is selected from the candidate skill-generation methods, namely the Skill Refiner and the DB Summarizer. Therefore, the table shows not only which method dominates overall, but also when the workflow switches from rewriting skills from newly collected records to refining already established skill files.

After iteration 1, the DB Summarizer wins almost every contest across EvoDRC, Ablation 1, and Ablation 3. At this stage, the newly collected repair records provide the first target-specific evidence for revising the skill files, and the DB Summarizer converts these records into updated repair knowledge, similar to how an engineer

Table 5: DRV count after each repair iteration for EvoDRC and the three ablation studies.

Iter.	Block1				Block2				Block3				Block4				Block5				Block6				Block7			
	E.	A.1	A.2	A.3	E.	A.1	A.2	A.3	E.	A.1	A.2	A.3	E.	A.1	A.2	A.3	E.	A.1	A.2	A.3	E.	A.1	A.2	A.3	E.	A.1	A.2	A.3
0		244				68				89				147				68				247				765		
1	145	208	145	192	29	30	29	43	14	12	14	62	125	96	125	140	38	42	38	58	78	153	78	159	394	675	394	759
2	233	201	198	165	18	26	18	30	5	4	6	51	112	87	77	89	39	35	27	47	49	137	73	66	374	613	368	759
3	217	198	124	122	14	37	17	29	2	2	0	21	112	63	43	61	4	25	20	30	38	128	47	48	353	588	355	758
4	129	199	124	115	16	26	17	18	0	0	—	0	101	56	36	42	2	19	4	16	23	128	23	36	284	602	357	684
5	109	207	134	96	8	61	13	18	—	—	—	—	22	56	34	15	0	17	2	16	18	118	20	0	275	440	358	684
Δ	-135	-37	-110	-148	-60	-7	-55	-50	-89	-89	-89	-89	-125	-91	-113	-132	-68	-51	-66	-52	-229	-129	-227	-247	-490	-325	-407	-81
~%	-55.3	-15.2	-45.1	-60.7	-88.2	-10.3	-80.9	-73.5	-100	-100	-100	-100	-85.0	-61.9	-76.9	-89.8	-100	-75.0	-97.1	-76.5	-92.7	-52.2	-91.9	-100	-64.1	-42.5	-53.2	-10.6

E. = EvoDRC, A.1–A.3 = Ablation 1–Ablation 3. Green numbers indicate the largest DRV count reduction for each block. the Dash marks iterations skipped by early stop after reaching zero DRVs.

Table 6: EvoDRC LLM cost (\$) per agent per each block [28].

Block	Repair	Skill Refiner	DB Summarizer	Skill Judge	Total
Block1	144.57	9.33	10.85	3.01	167.76
Block2	110.41	6.99	7.08	2.14	126.63
Block3	50.45	7.11	6.26	1.97	65.79
Block4	109.90	5.83	6.66	1.37	123.76
Block5	65.71	9.04	6.69	2.69	84.14
Block6	102.53	9.85	10.17	2.71	125.26
Block7	411.76	13.70	25.29	8.14	458.89
Total	995.33	61.85	73.00	22.03	1152.23
%	86.4%	5.4%	6.3%	1.9%	100.0%

Costs follow the Sonnet 4.6 API pricing per million tokens [28]: \$3 base input, \$3.75 five-minute cache write, \$6 one-hour cache write, \$0.30 cache read, and \$15 output.

Table 7: Majority skill file evolution winner across all metal layers for each block and iteration.

Iter	EvoDRC							Ablation 1							Ablation 3						
	1	2	3	4	5	6	7	1	2	3	4	5	6	7	1	2	3	4	5	6	7
1	D	D	S	D	D	D	D	D	D	D	D	D	D	D	D	D	D	D	D	D	D
2	D	T	S	D	T	T	D	D	D	S	S	S	S	D	D	S	D	D	S	D	—
3	S	S	S	—	T	S	S	D	S	T	S	S	S	D	D	D	D	S	T	D	D
4	S	S	D	S	D	S	D	S	T	S	S	S	S	T	S	S	D	D	S	S	D

S = Skill Refiner, D = DB Summarizer; T = Tie, — = no layer updated.

Table 8: Skill content evolution over iterations. Each row summarizes the content in the skill files for a repair iteration.

Iter	Block3				Block5				Block7			
	Cite	Sit.	Sug.	Warn	Cite	Sit.	Sug.	Warn	Cite	Sit.	Sug.	Warn
1	0	25	24	46	0	25	24	46	0	25	24	46
2	132	44	42	49	96	67	20	54	421	56	33	47
3	207	38	42	56	230	70	28	67	498	77	31	57
4	257	43	43	49	314	90	34	75	536	105	44	72
5	—	—	—	—	405	95	33	75	525	108	62	81

Cite: repair record citation count. Sit.: # of unique situations defined by rule and layer geometry context. # of Sug.: repair suggestions across situations. Warn: # of warnings across situations.

would draft a target-specific notebook from measured outcomes. From iteration 3 onward, the Skill Refiner, now operating on skill files that already contain target-specific knowledge, wins the majority of contests in all three configurations.

5.3 Skill Content Evolution

Each row in Table 8 summarizes the skill files across all layers used in each iteration for Block3, Block5, and Block7². A *situation* is one DRC rule encountered in one unique layout geometry context, e.g., enclosure violation under a specific metal routing pattern, a *suggestion* is one repair technique offered for a *situation*, a *warning* marks forbidden repair actions, and *cite* is the reference repair record in the Knowledge DB.

The CLA-derived initial skill files describe 25 situations with 24 repair suggestions. There is no cited record at iteration 1 since the DRC repair has just started. Skill evolution then adapts the content to each iteration’s repair trajectory. On Block5, the iteration-2 update prunes the suggestions from 24 to 20 after noticing the repair degradation, while situations and warnings keep growing to 95 and

²We show only results of three blocks for the interest of space. More skill files are available in [13]

Table 9: Iteration traces of the EvoDRC runs on Block 3, Block 5, and Block 7 over representative rule types.

Iter	M1.S.2			V2.M3.AUX.2			M5.AUX.3			V1.M1.EN.1		
	B3	B5	B7	B3	B5	B7	B3	B5	B7	B3	B5	B7
0	5	6	25	27	21	225	0	0	0	6	5	26
1	1	1	16	0	13	225	0	0	0	0	1	7
2	0	0	12	0	6	225	0	13	0	0	0	4
3	1	0	11	0	2	225	0	0	0	0	0	3
4	0	0	11	0	0	153	0	0	0	0	0	5
5	—	0	11	—	0	153	—	0	0	—	0	4

M1.S.2: minimum M1 tip-to-side spacing. V2.M3.AUX.2: V2 must match the M3 width perpendicular to the M3 direction. M5.AUX.3: M5 may not bend. V1.M1.EN.1: minimum enclosure of V1 by M1.

75, so the skill file becomes more specific about when a fix applies and what it must not break. On Block7, situation coverage grows fastest, from 25 to 108, matching its largest and most diverse violation set. On Block3, which keeps clearing violations, suggestions accumulate to 43 while warnings stay low. This explicitly shows how evolution adapts the skill to each case’s repair trajectory.

6 Discussion

Table 9 traces the EvoDRC runs on Block3, Block5, and Block7, whose repair trajectories are shown via four representative rule types³. M1.S.2 and V1.M1.EN.1 DRVs are cleaned in two iterations on Block3 and Block5, while Block7 keeps 11 of 25 M1.S.2 DRVs and 3 to 5 V1.M1.EN.1 DRVs after a first-iteration drop from 26 to 7. A repair introduces 13 M5.AUX.3 DRVs at iteration 2, degrading Block5 from 38 to 39 DRVs, and iteration 3 removes them with 90% of the remainder. One iteration clears the 27 V2.M3.AUX.2 DRVs on Block3, and four iterations clear the 21 on Block5, while Block7 stays at 225 until a coordinated repair at iteration 4 clears 72.

Overall, the DRV count reaches zero on Block3 and Block5 within four iterations, while Block7 retains several residual DRVs, with any intermediate increases recovering within two iterations. This trend demonstrates the advantage of EvoDRC. Every repair operation, including those that temporarily increase the DRV count, is evaluated and recorded, allowing the agent to explore the consequences of its actions rather than a greedy search path. The knowledge evolved from these repair records captures the resulting collateral effects and enables more coordinated repairs in subsequent iterations.

7 Conclusion

This paper presents EvoDRC, an agentic framework for post-route DRC repair. EvoDRC decomposes full-block layouts into bounded repair crops, repairs each crop with a four-role agent workflow, and records repair results. It further evolves layer-wise skill files through two paths, generated by the Skill Refiner agent and the DB Summarizer agent, respectively, and then selected by the Skill Judge agent. On DAC26-DRC-Benchmark, EvoDRC achieves an average DRV reduction of 83.6%.

³We show only four rule types and three blocks for the interest of space. More information is available in [13]

References

- [1] “How Qualcomm Got Faster Signoff DRC Convergence.” <https://semiengineering.com/how-qualcomm-got-faster-signoff-drc-convergence/>.
- [2] S. Yao, *et al.*, “ReAct: Synergizing Reasoning and Acting in Language Models,” in *Proc. ICLR*, 2023.
- [3] U. Sharma, *et al.*, “OpenROAD-Assistant: An Open-Source Large Language Model for Physical Design Tasks,” in *Proc. MLCAD*, 2024.
- [4] B.-Y. Wu, *et al.*, “OpenROAD Agent: An Intelligent Self-Correcting Script Generator for OpenROAD,” in *Proc. ICLAD*, 2025.
- [5] A. Ghose, *et al.*, “ORFS-agent: Tool-Using Agents for Chip Design Optimization,” in *Proc. MLCAD*, 2025.
- [6] H. Wu, *et al.*, “ChatEDA: A Large Language Model Powered Autonomous Agent for EDA,” *IEEE T. Comput. Aid. D.*, vol. 43, no. 10, p. 3184–3197, 2024.
- [7] V. A. Chhabria, *et al.*, “Generative Methods in EDA: Innovations in Dataset Generation and EDA Tool Assistants,” in *Proc. ICCAD*, 2024.
- [8] Z. Jiang, *et al.*, “CAPO: Certification-Guided Agentic Workflow for Physical Design Parameter Optimization,” in *Proc. GLSVLSI*, 2026.
- [9] C.-C. Chang, *et al.*, “DRC-Coder: Automated DRC Checker Code Generation Using LLM Autonomous Agent,” in *Proc. ISPD*, 2025.
- [10] B.-Y. Wu, *et al.*, “Special Research Session: Toward Agentic Solution for DRC Challenges in Digital VLSI Design,” in *Proc. DAC*, 2026.
- [11] P. Liu, *et al.*, “Bridging the Last Mile of Circuit Design: PostEDA-Bench, a Hierarchical Benchmark for PPA Convergence and DRC Fixing,” *arXiv preprint arXiv:2605.06936*, 2026.
- [12] “DAC26_DRC_Benchmark.” https://github.com/ASU-VDA-Lab/DAC26_DRC_Benchmark.
- [13] “EvoDRC.” <https://github.com/ASU-VDA-Lab/EvoDRC>.
- [14] A. Kahng, *et al.*, “Placement Tomography-Based Routing Blockage Generation for DRV Hotspot Mitigation,” in *Proc. ICCAD*, 2025.
- [15] S. Kim, *et al.*, “Methodology of Resolving Design Rule Checking Violations Coupled with Fully Compatible Prediction Model,” in *Proc. ISPD*, 2024.
- [16] Z. Xie, *et al.*, “RouteNet: Routability Prediction for Mixed-Size Designs Using Convolutional Neural Network,” in *Proc. ICCAD*, 2018.
- [17] A. B. Kahng, *et al.*, “TritonRoute: The Open-Source Detailed Router,” *IEEE T. Comput. Aid. D.*, vol. 40, no. 3, pp. 547–559, 2021.
- [18] Z. Qi, *et al.*, “Effective and Efficient Detailed Routing with Adaptive Rip-up Scheme and Pin Access Refinement,” in *Proc. GLSVLSI*, 2022.
- [19] C.-K. Cheng, *et al.*, “CoRe-ECO: Concurrent Refinement of Detailed Place-and-Route for an Efficient ECO Automation,” in *Proc. ICCD*, 2021.
- [20] J.-W. Jeon, *et al.*, “SLO-ECO: Single-Line-Open Aware ECO Detailed Placement and Detailed Routing Co-Optimization,” in *Proc. ISQED*, 2024.
- [21] “PostEDA-Bench.” <https://github.com/pengjas/posteda-bench>.
- [22] L. T. Clark, *et al.*, “ASAP7: A 7-nm FinFET Predictive Process Design Kit,” *Microelectronics Journal*, vol. 53, pp. 105–115, 2016.
- [23] “KLayout.” <https://www.klayout.de/>.
- [24] “OpenCores.” <https://opencores.org/>.
- [25] “OpenROAD-flow-scripts.” <https://github.com/the-openroad-project/openroad-flow-scripts>.
- [26] “Claude 4.6 Sonnet.” <https://www.anthropic.com/news/claude-sonnet-4-6>.
- [27] “Anthropic.” <https://www.anthropic.com/>.
- [28] “Claude Models & Pricing.” <https://platform.claude.com/docs/en/about-claude/pricing>.